

Contents

guide ReAct: Synergizing Reasoning and Acting in Language Models	1
0. 先给结论	2
1. 历史语境	2
2. ReAct 不是普通 CoT	3
3. 论文形式化机制	3
4. dense thought 和 sparse thought	4
5. ReAct 的最小系统图	5
6. 知识密集任务设置	5
7. 知识任务 prompt 和 baselines	6
8. 知识任务主结果	7
9. 成功和失败模式	7
10. Fine-tuning 结果	8
11. 决策任务: ALFWorld	9
12. ALFWorld 结果	9
13. 决策任务: WebShop	10
14. ReAct-IM 消融	10
15. 可解释性和人类纠错	11
16. Ethics 和安全边界	11
17. 论文没有证明什么	12
18. 和本仓库代码的对应关系	12
19. 本仓库 capstone 怎么读	13
20. Plan-and-Execute 与 ReAct 的区别	14
21. Reflexion 与 ReAct 的关系	14
22. Router 和 StateGraph	14
23. 本仓库最小实验 1: 观察是否真的影响下一步	15
24. 本仓库最小实验 2: 删掉 thought	15
25. 本仓库最小实验 3: 工具失败恢复	15
26. 本仓库最小实验 4: 加 Trace 审计	16
27. 用 AI agent 学 ReAct 的正确方式	16
28. 读完必须能回答	16
29. 学习节奏建议	17

guide_ReAct: Synergizing Reasoning and Acting in Language Models

原论文: ReAct: Synergizing Reasoning and Acting in Language Models

本地原文 PDF: 已入库, 同目录文件 01_react_synergizing_reasoning_and_acting.pdf

作者: Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao

会议版本: ICLR 2023

本 guide 目标: 让你真正理解现代 agent loop 的最小原型。读完后, 你应该能解释为什么 ReAct 不是“让模型多写几句思考”, 而是把 thought、action、observation 放进同一个可检查、可纠错、可执行的轨迹里。

0. 先给结论

ReAct 的核心可以压成一句话:

让语言模型交替生成 reasoning traces 和 task-specific actions, 让 reasoning 帮助模型计划、记忆、修正行动, 让 action 从外部环境拿到新 observation, 反过来约束下一轮 reasoning.

它的范式是:

Thought 1: 先想清楚要查什么或做什么

Action 1: 调工具或操作环境

Observation 1: 环境返回结果

Thought 2: 根据 observation 更新计划

Action 2: 继续查、点、拿、放、计算、结束

Observation 2: 新结果

...

Final Answer: 最终答案

今天几乎所有基础 agent loop 都能看到 ReAct 的影子:

- tool calling 里的 thought-action-observation.
- browser agent 的观察、点击、回看。
- coding agent 的计划、改文件、跑测试、看错误。
- RAG agent 的搜索、阅读、再搜索。
- state machine agent 的状态更新和下一步决策。

ReAct 的价值不是提出复杂算法, 而是提出一个足够简单、足够通用的交互格式: **模型可以边想边做, 做完再看, 看到以后再想。**

1. 历史语境

ReAct 出现前, LLM 推理和 LLM 行动大体是两条线。

一条线是 Chain-of-Thought, 也就是 CoT。CoT 让模型写出中间推理过程, 擅长算术、常识、多步文本推理。问题是它主要依赖模型内部知识:

- 它不能主动查外部资料。
- 它不能确认事实是否过时。
- 它一旦早期推理错了, 后面容易 error propagation。
- 它可能写出很顺的 hallucinated facts。

另一条线是 action generation。模型直接生成动作, 比如搜索、点击、移动、拿取物体、调用 API。问题是:

- 它不一定有显式计划。
- 它可能只做局部反应。
- 它难以追踪高层目标和子目标。
- 它的行为不容易诊断。

ReAct 的动机就是把这两条线接起来:

CoT:

good at internal reasoning
weak at grounding and interaction

Act-only:

good at interacting with environment
weak at high-level reasoning and memory

ReAct:
reason to act
act to reason

论文用人类做菜的例子说明这一点。人做菜时不会先把完整计划一次性写完，也不会盲目连续执行动作。人会想“现在都切好了，该烧水”，会检查冰箱，会发现没有盐后调整方案，会查菜谱。这种内心语言和外部行动的交替，就是 ReAct 想让 LLM 具备的能力。

2. ReAct 不是普通 CoT

普通 CoT 的轨迹可能是：

Question: Which magazine started first, Arthur's Magazine or First for Women?
Thought: Arthur's Magazine started in 1844. First for Women started in 1989.
Therefore Arthur's Magazine started first.
Answer: Arthur's Magazine.

如果这些日期来自模型记忆，它可能对，也可能错。你看不到模型有没有查过资料。

ReAct 的轨迹会像：

Question: Which magazine started first, Arthur's Magazine or First for Women?

Thought 1: I need to search Arthur's Magazine and First for Women.

Action 1: Search[Arthur's Magazine]

Observation 1: Arthur's Magazine was published from 1844 to 1846.

Thought 2: Arthur's Magazine started in 1844. I need First for Women.

Action 2: Search[First for Women]

Observation 2: First for Women is a magazine started in 1989.

Thought 3: 1844 is earlier than 1989.

Action 3: Finish[Arthur's Magazine]

区别在于：

- CoT 的事实可能来自内部记忆。
- ReAct 的事实可以来自 environment observation。
- CoT 的 reasoning trace 主要是解释。
- ReAct 的 reasoning trace 会决定下一次 action。
- ReAct 的 observation 又会改变下一次 reasoning。

这就是闭环。

3. 论文形式化机制

论文第 2 节给了一个简单形式化。

普通 agent 在时间步 t 看到 observation，然后根据上下文选择 action：

```
context_t = (obs_1, action_1, ..., obs_t)
action_t = policy(context_t)
```

ReAct 把 action space 扩展了。原本 action space 是环境动作集合 A ，例如搜索、点击、移动、拿物品。ReAct 增加一个 language thought space L :

$$A_{\text{hat}} = A + L$$

也就是说，模型下一步可以选择:

1. 生成一个环境 action，改变外部世界并拿到 observation。
2. 生成一个 language thought，不改变外部世界，但把有用信息写入上下文，帮助后续 action。

如果模型生成 thought:

```
thought_t in L
environment unchanged
context_{t+1} = context_t + thought_t
```

如果模型生成 action:

```
action_t in A
environment executes action_t
observation_{t+1} returned
context_{t+1} = context_t + action_t + observation_{t+1}
```

这就解释了为什么 thought 不是装饰文字。thought 是上下文状态的一部分，是后续 policy 的输入。

4. dense thought 和 sparse thought

ReAct 在不同任务中使用 thought 的密度不同。

在 HotpotQA、FEVER 这类知识密集推理任务里，论文采用 dense thought-action-observation:

```
Thought 1
Action 1
Observation 1
Thought 2
Action 2
Observation 2
...
```

原因是每一步检索都需要明确解释:

- 我要查哪个实体。
- observation 是否包含答案。
- 如果没有，下一步该换哪个 query。
- 最后如何合成答案。

在 ALFWorld、WebShop 这类长 horizon 决策任务里，论文允许 sparse thought。因为 agent 可能要做几十个动作，不必每个动作前都长篇 reasoning。关键是在重要位置插入 thought:

- 分解高层目标。
- 判断子目标是否完成。
- 决定下一个子目标。
- 用常识推断物品可能在哪里。
- 在观察不符合预期时调整计划。

这对工程很重要。真实 agent 不是 thought 越多越好。thought 太少会盲动，thought 太多会浪费上下文、增加错误表述和成本。ReAct 的本质是让 thought 出现在能改变 action 的位置。

5. ReAct 的最小系统图

可以把 ReAct 画成:

```
question or goal
  ->
prompt with examples
  ->
LLM generates Thought
  ->
LLM generates Action
  ->
tool or environment executes Action
  ->
Observation appended to context
  ->
LLM reads updated context
  ->
next Thought or Final Answer
```

注意两条信息流:

Reason to act:

Thought tells the agent what action is useful next.

Act to reason:

Observation gives new facts so the next Thought is grounded.

少任何一边都会退化:

- 只有 reason: 容易 hallucinate。
- 只有 act: 容易没有计划。
- 有 reason 但不用 observation: 是伪 ReAct。
- 有 action 但 observation 不进上下文: 工具调用对模型没有反馈。

6. 知识密集任务设置

论文第 3 节测试两个任务:

HotpotQA。

多跳问答, 需要跨两个或更多 Wikipedia passages 推理。

FEVER。

事实验证任务。给一个 claim, 输出 SUPPORTS、REFUTES 或 NOT ENOUGH INFO。

论文采用 question-only setup: 模型只拿到问题或 claim, 没有直接给 supporting paragraphs。模型必须依靠内部知识, 或者通过外部 Wikipedia API 检索。

Wikipedia API 有三个动作:

Search[entity]

如果页面存在, 返回该页面前 5 句。

如果不存在, 返回相似实体建议。

Lookup[string]

在当前页面中查找包含 string 的下一句，模拟 Ctrl+F。

Finish[answer]

用 answer 结束任务。

这个 action space 故意很弱。它不是强检索器，只能模拟人类比较朴素地查 Wikipedia。这样可以逼模型显式 reasoning:

- 搜什么实体。
- 查不到时怎么换搜索词。
- 页面里缺信息时查什么关键词。
- 多段信息怎么合成答案。

7. 知识任务 prompt 和 baselines

ReAct prompting:

- HotpotQA 随机选 6 个训练样例，手写 ReAct trajectory。
- FEVER 随机选 3 个训练样例，手写 ReAct trajectory。
- 每个 trajectory 包含 thought-action-observation steps。
- thought 用于分解问题、提取 observation、做常识或算术、重写 query、合成答案。

论文对比的 baselines:

Standard.

去掉 thought、action、observation，只保留普通 few-shot answer。

CoT.

去掉 action 和 observation，只保留 reasoning traces。

CoT-SC.

CoT self-consistency. 采样 21 条 CoT trajectory，temperature = 0.7，用多数答案。

Act.

去掉 thought，只保留 action 和 observation。它像一个能查网页的行动模型，但没有显式 reasoning。

ReAct.

保留 thought、action、observation。

此外论文提出两种组合方法:

ReAct -> CoT-SC.

如果 ReAct 在限定步数内没有返回答案，就退回 CoT-SC。HotpotQA 限 7 步，FEVER 限 5 步。

CoT-SC -> ReAct.

如果 CoT-SC 多数答案不够强，例如 n 个样本里多数答案少于 n/2，说明内部知识不自信，就退回 ReAct 检索外部知识。

这点非常关键: 论文没有说 ReAct 永远替代 CoT。它说内部知识和外部检索各有优势，好的系统要知道什么时候切换。

8. 知识任务主结果

PaLM-540B prompting 结果:

HotpotQA exact match:

- Standard: 28.7
- CoT: 29.4
- CoT-SC: 33.4
- Act: 25.7
- ReAct: 27.4
- CoT-SC -> ReAct: 34.2
- ReAct -> CoT-SC: 35.1
- Supervised SoTA: 67.5

FEVER accuracy:

- Standard: 57.1
- CoT: 56.3
- CoT-SC: 60.4
- Act: 58.9
- ReAct: 60.9
- CoT-SC -> ReAct: 64.6
- ReAct -> CoT-SC: 62.0
- Supervised SoTA: 89.5

这些数字要细读。

第一, ReAct 比 Act 好。HotpotQA 是 27.4 vs 25.7, FEVER 是 60.9 vs 58.9。说明 thought 对 action 有帮助, 尤其是最终答案合成。

第二, ReAct 不总是单独赢 CoT。HotpotQA 上 CoT 是 29.4, ReAct 是 27.4; FEVER 上 ReAct 是 60.9, CoT 是 56.3。

第三, 组合最好。HotpotQA 上 ReAct -> CoT-SC 到 35.1, FEVER 上 CoT-SC -> ReAct 到 64.6。

直觉是:

- HotpotQA 常需要结构化多跳推理, CoT 的推理结构有优势。
- FEVER 对事实精确性敏感, 外部检索能减少 hallucination。
- ReAct 可以 grounding, 但受搜索结果质量影响。
- CoT 可以流畅推理, 但容易编事实。

所以最佳策略不是“永远 ReAct”, 而是“会查、会想、会在不自信时切换”。

9. 成功和失败模式

论文人工分析了 HotpotQA 上 ReAct 和 CoT 的成功/失败模式。随机采样 ReAct 与 CoT 各自正确和错误轨迹, 共 200 个例子。

成功样本中:

- True positive: ReAct 94%, CoT 86%。
- False positive: ReAct 6%, CoT 14%。

这里 false positive 指答案对了, 但推理 trace 或事实有 hallucination。ReAct 更低, 说明外部 observation 让轨迹更 grounded。

失败样本中:

- Reasoning error: ReAct 47%, CoT 16%。
- Search result error: ReAct 23%, CoT 不适用。
- Hallucination: ReAct 0%, CoT 56%。
- Label ambiguity: ReAct 29%, CoT 28%。

这组数字很有教育意义。

ReAct 的强项:

- 减少 hallucination。
- 轨迹更可检查。
- 事实更 grounded。

ReAct 的弱项:

- 检索结果不好时会被带偏。
- 结构化 thought-action 约束会降低推理灵活性。
- 有时会重复前面的 thought/action, 跳不出循环。

CoT 的强项:

- 内部推理连贯。
- 结构化多跳逻辑有时更顺。

CoT 的弱项:

- 容易编事实。
- 事实错了仍能写出看起来合理的推理。

读到这里, 你应该形成一个成熟判断:

工具调用不是免费午餐。它减少纯内存 hallucination, 但会引入 retrieval failure、tool error、looping 和上下文管理问题。

10. Fine-tuning 结果

论文还做了初步 fine-tuning 实验。

问题是: 手写大量 ReAct trajectories 很贵。作者用 bootstrap 方法, 收集 3000 条由 ReAct 生成且最终答案正确的 trajectories, 用它们 fine-tune PaLM-8B 和 PaLM-62B, 让模型根据输入问题生成完整 trajectory。

结果的核心观察:

- 在纯 prompting 下, PaLM-8B/62B 的 ReAct 表现较差, 因为要从少量 in-context examples 学会 reasoning 和 acting 都很难。
- 但 fine-tune 后, ReAct 成为 Standard、CoT、Act、ReAct 四种方法中最强。
- PaLM-8B fine-tuned ReAct 超过所有 PaLM-62B prompting 方法。
- PaLM-62B fine-tuned ReAct 超过所有 PaLM-540B prompting 方法。

为什么? 论文的解释是:

- Fine-tuning Standard/CoT 更像教模型记忆可能会 hallucinate 的知识事实。
- Fine-tuning Act/ReAct 是教模型如何行动、如何查 Wikipedia、如何根据 observation 更新。
- 这种技能更可泛化。

这对现代 agent 很重要。只用 prompt 可以快速搭原型，但高质量 agent 轨迹数据非常有价值。后来的 tool-use fine-tuning、trajectory imitation、self-reflection 数据，都是沿着这个方向发展。

11. 决策任务: ALFWorld

论文第 4 节转向 interactive decision making。

ALFWorld 是一个文本游戏，和 embodied ALFRED benchmark 对齐。Agent 需要在模拟家庭环境中完成任务，例如：

- 找到 paper。
- 打开或使用 desklamp。
- 清洗某个物品。
- 加热、冷却、拿取、放置物体。

任务特点：

- 可能有 50 多个地点。
- expert policy 可能需要 50 多步。
- reward sparse，只有完成目标才成功。
- 需要常识，比如 desklamp 可能在 desk、shelf、dresser 附近。

ReAct prompt:

- 每个 task type 标注 3 条训练轨迹。
- 每条轨迹有 sparse thoughts。
- thought 用于分解目标、追踪子目标、决定下一子目标、用常识推断物品位置。
- 对每个 task type 构造 6 个 prompts，即从 3 条轨迹里取 2 条的不同排列。
- 评测 134 个 unseen games。

Act baseline 用相同轨迹但去掉 thoughts。这是一个很干净的对照：action 经验一样，差别在是否有 reasoning traces。

12. ALFWorld 结果

ALFWorld overall success rate:

- Act best of 6: 45
- ReAct average: 57
- ReAct best of 6: 71
- ReAct-IM average: 48
- ReAct-IM best of 6: 53
- BUTLER best of 8: 37
- BUTLERg best of 8: 22

ReAct best of 6 的各 task type 成功率:

- Pick: 92
- Clean: 58
- Heat: 96
- Cool: 86
- Look: 78
- Pick 2: 41
- Overall: 71

论文强调: ReAct best of 6 显著超过 Act best of 6 和 BUTLER。更有意思的是, ReAct 最差 trial 的 overall 也有 48, 仍高于 Act best of 6 的 45 和 BUTLER 的 37。

为什么 thought 有帮助?

- Act-only 容易忘记高层目标。
- Act-only 不会明确追踪子目标是否完成。
- Act-only 遇到没找到物品时容易盲目继续。
- ReAct thought 可以说“我已经清洗完了, 现在要把物品放到 countertop”。

这就是 reasoning in interactive tasks 的价值。

13. 决策任务: WebShop

WebShop 是更接近真实网页的环境:

- 1.18M real-world products.
- 12k human instructions.
- 用户要求通常包含多个属性, 例如价格、颜色、用途、尺寸。
- agent 需要搜索商品、点击商品、选择选项、购买。
- 测试集 500 条 instructions.

指标:

- Score: 商品满足目标属性的平均比例。
- Success rate: 商品满足全部要求的 episode 比例。

结果:

- Act: score 62.3, success rate 30.1.
- ReAct: score 66.6, success rate 40.0.
- IL: score 59.9, success rate 29.1.
- IL+RL: score 62.4, success rate 28.7.
- Human expert: score 82.1, success rate 59.6.

ReAct 比此前最好 success rate 高约 10 个百分点。原因是网页环境噪声大, 商品标题、描述、选项非常多。Thought 可以把用户要求和页面 observation 对齐:

```
Thought: The instruction asks for a blue 16-pack apple cinnamon snack.
This product is strawberry banana, so it does not match.
Action: go back or click another result.
```

没有 thought 的 Act 可能看到一个相关产品就买, 漏掉口味、数量或尺寸限制。

但 ReAct 仍远低于 human expert。人类会更积极地重写 query、比较多个产品、检查更多页面。论文也没有说 ReAct 已经解决 web navigation。

14. ReAct-IM 消融

论文还对比 ReAct 和 ReAct-IM。IM 指 Inner Monologue 风格。

ReAct-IM 的 thought 更像环境反馈复述, 例如:

```
I see the object is here.
I still need to complete the task.
```

它缺少 ReAct 那种灵活的高层 reasoning:

- 分解目标。
- 判断子目标完成。
- 推断物品可能位置。
- 计划下一段行动。

ALFWorld 中:

- ReAct best overall 71。
- ReAct-IM best overall 53。

这说明不是任何“内心独白”都足够。真正有用的 thought 必须改变策略,而不是机械复述 observation。

15. 可解释性和人类纠错

ReAct 的另一个贡献是可诊断性。

因为轨迹中有 thought、action、observation, 人可以检查:

- 模型是基于内部知识还是外部 observation。
- 哪一步搜索错了。
- 哪一步 thought 误读了 observation。
- 哪一步 action 不符合当前目标。
- 是否陷入重复。

论文 Appendix A.3 展示 human-in-the-loop behavior correction。人类只需编辑轨迹中的少数 thought, 例如删除 hallucinated sentence 或加入提示, 就能显著改变后续行动, 让 ALFWorld trajectory 成功。

这对 agent 学习很重要。你和 AI agent 协作时, 不应该只看最终答案。你要学会看 trace:

这个 action 是从哪个 thought 来的?

这个 thought 有没有引用 observation?

如果 observation 错了, 下一步有没有恢复?

如果工具失败了, agent 有没有换策略?

16. Ethics 和安全边界

ReAct 让模型能行动, 所以风险比纯文本更高。

论文 ethics statement 提到:

- LLM 接入 action space 后, 可能查到不合适或隐私信息。
- 如果连接真实网页、真实购买、真实物理环境, 可能执行有害动作。
- 实验中限制 action space, 避免危险能力。
- Wikipedia 和 WebShop benchmark 不包含真实购买能力, 也不能编辑 Wikipedia。

这说明 ReAct 只是一种 agent loop 形式, 不自动安全。真实系统必须额外设计:

- 工具权限边界。
- 人类确认。
- 沙箱。
- action allowlist。
- 速率限制。
- trace logging。
- 回滚机制。
- 对敏感动作的审批。

不要因为 ReAct trace 看起来可解释，就以为系统安全。可解释只是更容易诊断，不是自动无害。

17. 论文没有证明什么

这篇论文没有证明:

- ReAct 永远优于 CoT。
- thought 越多越好。
- few-shot ReAct 能解决所有复杂 agent 任务。
- 工具调用能消除 hallucination。
- 搜索到的 observation 一定可靠。
- ReAct trace 一定忠实反映模型内部原因。
- 连接真实环境后不需要安全控制。

论文自己也承认:

- 复杂任务和大 action space 需要更多 demonstrations。
- in-context learning 会受到上下文长度限制。
- prompting 下的 ReAct 有时难学。
- fine-tuning 更有潜力，但需要高质量轨迹数据。
- ReAct 可以和 RL、multi-task training、human feedback 结合。

成熟读法是: ReAct 是 agent loop 的最小范式，不是完整产品系统。

18. 和本仓库代码的对应关系

本模块最重要文件:

- learning/agent-foundations/src/react_loop.py
- learning/agent-foundations/src/common.py
- learning/agent-foundations/src/tools/search_mock.py
- learning/agent-foundations/src/tools/calculator.py
- learning/agent-foundations/src/capstone_react.py
- learning/agent-foundations/src/reflexion_demo.py
- learning/agent-foundations/src/plan_execute.py
- learning/agent-foundations/src/router_pattern.py
- learning/agent-foundations/src/state_machine.py
- learning/agent-foundations/src/tracing.py

common.py 提供 agent 基础数据结构:

- Tool: 工具名、描述、schema、执行函数。
- ActionResult: 工具执行结果，可以转成 observation。
- Step: 一步 thought-action-observation。
- Trace: 完整轨迹、final answer、token 估算。
- parse_action: 从模型输出解析 Action: tool(args)。
- parse_final: 从模型输出解析 Final Answer。
- make_pattern_llm: toy mock LLM，用规则模拟模型输出。

react_loop.py 是论文最小机制:

```
for step in range(1, max_steps + 1):
    out = llm(history + f"Thought {step}:")
    final = parse_final(out)
```

```

if final is not None:
    return trace

name, args = parse_action(out)
result = tools[name].func(args)
obs = result.to_obs()
trace.add(...)
history += f"\n{out}\nObservation {step}: {obs}\n"

```

这段代码对应:

```

LLM output thought/action
->
tool executes action
->
observation appended to history
->
LLM sees new history
->
next step

```

本地 self-test 已改成 Observation-gated mock: 第二步必须看到上一轮 observation 中的结果, 才会输出 final。这更贴近 ReAct 精髓。

19. 本仓库 capstone 怎么读

capstone_react.py 的任务是:

```

Find the 2025 most popular LLM name,
then compute its name_length times 3.

```

这是 toy mock, 不是真实互联网事实。search_mock 返回:

```
Claude (mock result; 6-character name for capstone arithmetic)
```

轨迹:

```

Thought 1: I need to find the LLM first.
Action 1: search_mock("2025 most popular LLM")
Observation 1: Claude ...

```

```

Thought 2: Claude has 6 characters, so 6*3.
Action 2: calculator("6 * 3")
Observation 2: 18

```

```

Thought 3: Got 18, final.
Final Answer: 18

```

这个小例子虽然简单, 但覆盖了 ReAct 的关键:

- 第一步不知道事实, 先搜索。
- 第二步根据 observation 决定计算。
- 第三步根据计算 observation 给 final。
- trace 可以输出成 Markdown。
- cost_summary 可以估算 token 和 tool calls。

20. Plan-and-Execute 与 ReAct 的区别

plan_execute.py 展示另一种 agent pattern:

```
planner writes all steps once
executor runs steps in order
final model summarizes
```

它适合计划比较稳定的任务。但它和 ReAct 不同:

- Plan-and-Execute 先规划后执行。
- ReAct 边执行边改计划。
- Plan-and-Execute 的 plan 可能过早锁死。
- ReAct 能根据 observation 调整下一步。

真实 coding agent 常常需要 ReAct 式循环, 因为测试失败、文件不存在、依赖缺失都会改变下一步。

21. Reflexion 与 ReAct 的关系

reflexion_demo.py 展示 Reflexion:

```
attempt 1:
  run ReAct
  evaluator says fail
  reflect on failure
  store lesson in memory
```

```
attempt 2:
  run ReAct with memory
  improve behavior
```

ReAct 是单次 episode 内的 thought-action-observation loop。Reflexion 是跨 episode 的 verbal memory loop。

可以这样分:

ReAct:

```
within one attempt, use observations to choose next action.
```

Reflexion:

```
after a failed attempt, write lesson and use it next attempt.
```

你学 agent 时要把这两个 loop 分清楚。

22. Router 和 StateGraph

router_pattern.py 展示 routing:

- LLM router 根据问题选择 math、code、chat 等 handler。
- embedding_router 根据问题向量和 handler 描述相似度选择 handler。

这对应更复杂 agent 系统里的 orchestration。ReAct 是一个 loop, router 是选择哪个 loop 或工具链。

state_machine.py 展示 LangGraph-style StateGraph:

- nodes 是状态更新函数。

- edges 是固定转移。
- conditional edges 根据状态决定下一节点。
- END 结束。

现代 agent 工程常把 ReAct loop 放进 graph:

```
plan node
  -> act node
  -> observe node
  -> decide node
  -> finish or retry
```

这就是从论文范式走向生产框架的桥。

23. 本仓库最小实验 1: 观察是否真的影响下一步

打开 react_loop.py 的 _self_test。

你会看到第二条规则依赖:

```
r"Observation 1:[\s\S]*5[\s\S]*Thought 2:"
```

这表示 mock LLM 必须在 prompt history 里看到 Observation 1 和 5, 才会输出 final answer。

把工具 calculator 故意改错, 或者把第一步 action 改成 calculator(2+4), 你应该看到 final 不再是 5。这个实验会让你理解:

ReAct 的状态不是 prompt 开头的 question, 而是不断增长的 history。

24. 本仓库最小实验 2: 删掉 thought

把 capstone_react.py 中的第二步 thought 简化成:

```
Thought: continue.
Action 2: calculator("6 * 3")
```

功能上可能仍然通过, 但 trace 可解释性下降。然后再问自己:

- 这个 action 为什么是 6 * 3?
- 6 从哪里来?
- 是否引用了 observation?
- 如果 search result 改成 Gemini, 计算会不会改?

这就是 Act-only 和 ReAct 的差别。Action 可以碰巧对, 但没有 thought 时, 人很难审计策略。

25. 本仓库最小实验 3: 工具失败恢复

修改 search_mock.py, 让 popular llm 返回 no results。然后设计一个第二步:

```
Thought 2: Search failed. I should search a broader query.
Action 2: search_mock("popular AI model")
```

如果你的 loop 能处理这个恢复, 就更像论文中的 search reformulation。

这对应 HotpotQA/FEVER 里的场景:

- Search[Adam Clayton Powell] 找不到。
- 模型读 suggestions。

- 改搜 Adam Clayton Powell (film)。

ReAct 的一个优点就是把失败 observation 变成下一步 query reformulation。

26. 本仓库最小实验 4: 加 Trace 审计

跑:

```
python learning\agent-foundations\src\capstone_react.py
```

看输出的 Markdown trace。你应该检查:

1. 每个 action 前是否有 thought。
2. 每个 thought 是否引用了之前 observation。
3. final answer 是否来自最后 observation。
4. tool calls 是否符合预期。
5. 如果某一步失败, trace 是否能定位失败点。

这比只看 final answer 更像真实 agent debugging。

27. 用 AI agent 学 ReAct 的正确方式

不要让 agent 直接总结论文。你要让它考你, 并要求你对到代码。

推荐提示词:

我正在读 ReAct 论文。请一次只问一个问题。

问题必须覆盖:

1. CoT 和 Act-only 分别缺什么。
2. ReAct 如何把 action space 扩成 environment actions + language thoughts。
3. thought 为什么不直接改变环境但仍然重要。
4. HotpotQA/FEVER 里 ReAct、CoT、Act、CoT-SC 的差异。
5. ALFWorld/WebShop 为什么 sparse thought 有用。
6. ReAct 的失败模式和安全风险。

每次我回答后, 请要求我指出本仓库哪个函数或测试对应这个概念。

闭卷复述目标:

ReAct 是一种让 LLM 交替生成 reasoning traces 和 task-specific actions 的 agent 范式。

Thought 用来分解目标、追踪状态、调整计划; Action 用来搜索、操作环境或结束任务; Observation 把外部反馈在 HotpotQA 和 FEVER 中, ReAct 通过 Wikipedia API 降低 CoT 的 hallucination, 但也受检索失败和循环错在 ALFWorld 和 WebShop 中, ReAct 的 sparse reasoning 帮助长 horizon 决策, 显著超过 Act-only、IL 或它奠定了现代 tool-using agent loop 的基本形式, 但不是安全系统本身, 仍需要工具权限、沙箱、人类审计和高

28. 读完必须能回答

你应该能闭卷回答:

1. CoT 的主要问题是什么?
2. Act-only 的主要问题是什么?
3. ReAct 中 thought 和 action 的区别是什么?
4. 为什么 thought 不改变环境但会改变 agent 状态?
5. HotpotQA/FEVER 的三个 Wikipedia actions 是什么?
6. 为什么 ReAct 在 FEVER 上更明显超过 CoT?
7. 为什么 ReAct 在 HotpotQA 上不一定单独超过 CoT?

8. CoT-SC -> ReAct 和 ReAct -> CoT-SC 分别什么时候切换?
9. ReAct 的 hallucination 和 reasoning error 分别怎样变化?
10. ALFWorld 中 sparse thought 解决什么问题?
11. WebShop 中 thought 如何帮助选择商品?
12. ReAct-IM 为什么弱于 ReAct?
13. 本仓库 `react_loop.py` 哪一行把 observation 写回 history?
14. 本仓库 `capstone_react.py` 如何证明 Observation 影响下一步?
15. ReAct 接入真实工具时需要哪些安全控制?

29. 学习节奏建议

建议按三遍读:

第一遍只看机制:

- Figure 1。
- Section 2。
- 本 guide 第 2 到第 5 节。
- `react_loop.py`。

目标是画出 thought-action-observation loop。

第二遍看证据:

- Table 1 和 Table 2。
- ALFWorld Table 3。
- WebShop Table 4。
- 本 guide 第 8 到第 14 节。

目标是讲清楚 ReAct 相对 CoT、Act、ReAct-IM 的优缺点。

第三遍看工程:

- `capstone_react.py`。
- `tracing.py`。
- `plan_execute.py`。
- `state_machine.py`。

目标是把 ReAct 放进现代 agent 框架里, 而不是停留在论文 prompt 格式。

真正掌握的标志是: 你能看一段 agent trace, 指出每个 action 是不是由有效 thought 驱动, 每个 thought 有没有吸收 observation, 以及失败时应该改 prompt、工具、状态机还是安全边界。